Solid State Physics
Tight-Binding Model Project

Zengxuan Liu
3042027511

April 29, 2026

Contents

Chapter 1 **Simple 1D Tight-Binding Model** — Page 2
1.1 Task 1: Hamiltonian construction 2
1.2 Task 2: Eigenvalue problem 3

Chapter 2 **The Su–Schrieffer–Heeger (SSH) model** — Page 7
2.1 Task 1: Hamiltonian construction 7

Chapter 1

Simple 1D Tight-Binding Model

1.1 Task 1: Hamiltonian construction

Question 1

Write a program that constructs the Hamiltonian matrix in the periodic boundary conditions (PBC), where site N is identified with site 0.

Question 2

Modify the code to handle open boundary conditions (OBC), where sites 0 and $N-1$ are not connected.

Definition 1.1.1: Hamiltonian

The Hamiltonian for the 1D tight-binding model is given by:

$$H = -t \sum_n \Big(|n+1\rangle \langle n| + |n\rangle \langle n+1| \Big)$$

Solution: We are going to define the Hamiltonian matrix for the 1D tight-binding model under both periodic and open boundary conditions using the code block below:

```
# PBC
def pbc_Hamiltonian(n, t = 1):
    H = np.zeros((n, n))
    for i in range(n):
        H[i, (i + 1) % n] = -t
        H[(i + 1) % n, i] = -t
    return H

# OBC
def obc_Hamiltonian(n, t = 1):
    H = np.zeros((n, n))
    for i in range(n - 1):
        H[i, i + 1] = -t
        H[i + 1, i] = -t
    return H
```

Note:-

n is the number of lattice sites and t is the hopping parameter. The key to ensure periodic boundary conditions is to use modulo arithmetic to connect the first and last lattice sites.

Example 1.1.1 (Simple 1D Tight-Binding Model Hamiltonian in PBC and OBC)

```
# Example
N = 5
H_PBC = pbc_Hamiltonian(N)
H_OBC = obc_Hamiltonian(N)

print("Hamiltonian␣in␣PBC:")
print(H_PBC)
print(scipy.linalg.ishermitian(H_PBC))
print("Hamiltonian␣in␣OBC:")
print(H_OBC)
print(scipy.linalg.ishermitian(H_OBC))
```

whose outputs are as follows:

```
Hamiltonian in PBC:
[[ 0. -1.  0.  0. -1.]
[-1.  0. -1.  0.  0.]
[ 0. -1.  0. -1.  0.]
[ 0.  0. -1.  0. -1.]
[-1.  0.  0. -1.  0.]]
True
Hamiltonian in OBC:
[[ 0. -1.  0.  0.  0.]
[-1.  0. -1.  0.  0.]
[ 0. -1.  0. -1.  0.]
[ 0.  0. -1.  0. -1.]
[ 0.  0.  0. -1.  0.]]
True
```

1.2 Task 2: Eigenvalue problem

Question 1

Diagonalize your Hamiltonian for both boundary conditions for $N = 100$. It is convenient to use `numpy.linalg.eigh()`.

Solution: Using `np.linalg.eigh()` to diagonalize the Hamiltonian:

```
# Diagonalize
N = 100
E_PBC, psi_PBC = np.linalg.eigh(pbc_Hamiltonian(N))
E_OBC, psi_OBC = np.linalg.eigh(obc_Hamiltonian(N))
```

Note:-

The outputs of `np.linalg.eigh()` are arrays of eigenvalues and respective eigenvectors, and the arrays are sorted in ascending order of eigenvalues.

Question 2

Plot the energy spectrum of both the periodic and open systems on the same figure. How similar or different are they? Is there a degeneracy in the periodic spectrum? What is the origin of this degeneracy? Does the open system break or preserve this degeneracy? Why?

Solution: Plotting the energy spectrum:

```
plt.figure(figsize=(10, 6))
N = 100

plt.plot(E_PBC, 'o', label='PBC', markersize=5, color='blue')
plt.plot(E_OBC, 's', label='OBC', markersize=5, color='red')
plt.xlabel('Eigenvalue␣Index')
plt.ylabel('Eigenvalue')
plt.title('Eigenvalues␣of␣PBC␣and␣OBC␣Hamiltonians')
plt.legend(loc='upper␣left')
plt.grid(True, alpha = 0.3)
plt.xlim(0, N)
plt.show()
```

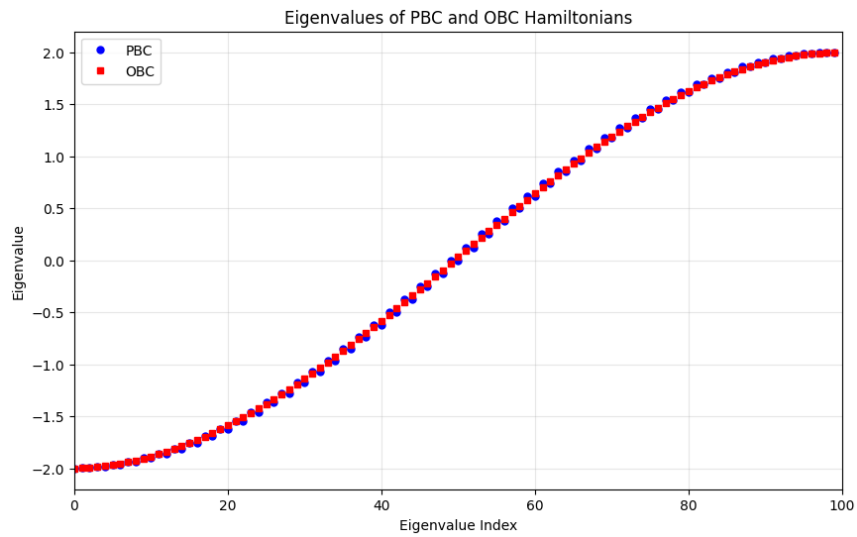

Figure 1.1: Eigenvalues of PBC and OBC Hamiltonians

Note:-

Consider the Hamiltonian and eigenvalue equation for a 1D tight-binding model:

$$H = -t \sum_{n=1}^{N} \Big(|n+1\rangle \langle n| + |n\rangle \langle n+1| \Big), \; H|\psi\rangle = E|\psi\rangle$$

where $|\psi\rangle = \sum_{n=1}^{N} \psi_n |n\rangle$. Furthermore, we can express the eigenequation as:

$$E\psi_n = -t(\psi_{n+1} + \psi_{n-1})$$

(a) PBC: As common choice for periodic system, we can use plane wave eigenstates, we can write:

$$\psi_n = \frac{1}{\sqrt{N}} e^{ikn}$$

where $k = \frac{2\pi m}{N}$ for $m = 0, 1, \ldots, N-1$. Thus, we can substitute this into the eigenequation and find the corresponding eigenvalues:

$$E(k) = -2t\cos(k) = -2t\cos\left(\frac{2\pi m}{N}\right)$$

which shows the degeneracy that $E(k) = E(-k)$.

(b) OBC: For the open boundary conditions, we can use standing wave eigenstates, we can write:

$$\psi_n = \frac{1}{\sqrt{N}} \sin\left(\frac{m\pi n}{N+1}\right)$$

where $m = 1, 2, \ldots, N$. Thus, we can substitute this into the eigenequation and find the corresponding eigenvalues:

$$E(m) = -2t \cos\left(\frac{m\pi}{N+1}\right)$$

- Both systems share the same energy spectrum, which can be observed from both the plot and the analytical expressions.
- There is a degeneracy in the periodic system because of periodic boundary conditions. This periodicity maintains the translational symmetry which ensures that each atom has the same neighborhood environment. Therefore, this symmetry leads to degenerate energy levels.
- The open system breaks this degeneracy because the boundary conditions are different, leading to a unique energy spectrum for each state.

Question 3

1. Plot the wave func1on probability distribu1on, $|\psi(x)|^2$, for the ground state and first two excited states of the **periodic** system as a func1on of posi1on and show them on the same plot. What do you no1ce about the first two excited states?
2. Plot the wave func1on probability distribu1on, $|\psi_m(x)|^2$, for the ground state and first two excited states of the **open** system as a func1on of posi1on and show them on the same plot. What do you notice about the number of nodes?

Solution: For 1D chains, the position can be represented by a discrete set of sites. Here, the position is indexed by n. Plotting the wave function probability distribution:

```
# Ground state and first two excited state probability plots
def plot_ground_and_excited_states(N, E, psi, title):
    x = np.arange(N)

    psi_gs = psi[:, 0]   # ground state
    psi_e1 = psi[:, 1]   # first excited state
    psi_e2 = psi[:, 2]   # second excited state

    plt.figure(figsize=(10, 6))
    plt.plot(x, np.abs(psi_gs)**2, label=f"{'Ground␣state':}␣E␣=␣{E[0]:.3f}", color="blue")
    plt.plot(x, np.abs(psi_e1)**2, label=f"{'1st␣excited␣state':}␣E␣=␣{E[1]:.3f}", color="orange")
    plt.plot(x, np.abs(psi_e2)**2, label=f"{'2nd␣excited␣state':}␣E␣=␣{E[2]:.3f}", color="red")

    plt.xlabel("Position␣index")
    plt.ylabel(r"$|\psi(x)|^2$")
    plt.xlim(0, N-1)
    plt.title(title)
    plt.legend(loc="upper␣right")
    plt.grid(True, alpha=0.3)
    plt.show()

plot_ground_and_excited_states(N, E_PBC, psi_PBC, "PBC␣probability␣distributions")
plot_ground_and_excited_states(N, E_OBC, psi_OBC, "OBC␣probability␣distributions")
```

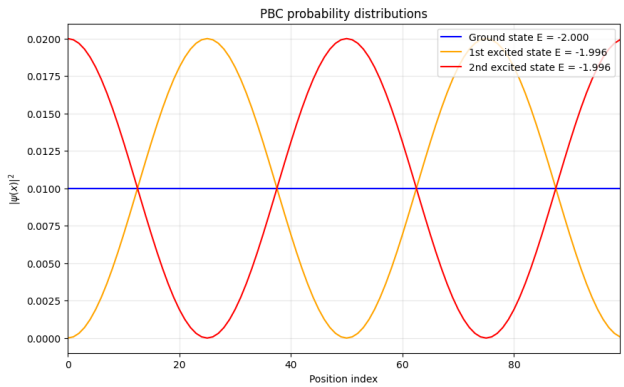

Figure 1.2: Probability distribution for PBC system

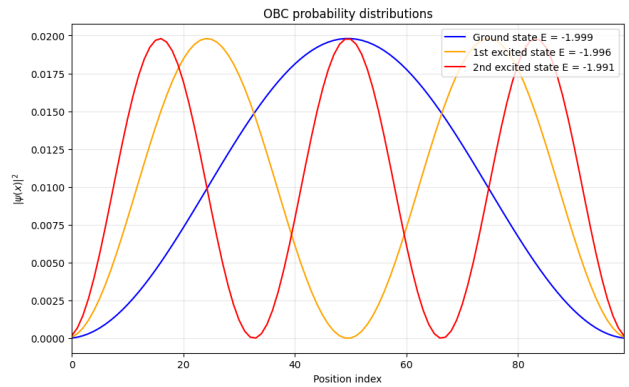

Figure 1.3: Probability distribution for OBC system

Note:-

1. PBC: The probability distribution for the ground state and first two excited states shows a periodic pattern due to the periodic boundary conditions. For the first two states, they are degenerate so their wave functions are identical despite an overall phase shift.

2. OBC: The number of nodes in the probability distribution for the ground state and first two excited states are 1, 3, and 5 respectively, increasing with the energy level.

Chapter 2

The Su–Schrieffer–Heeger (SSH) model

2.1 Task 1: Hamiltonian construction

Question 1

Write a program to construct the Hamiltonian matrix for this system in both periodic and open boundary conditions. Make sure that there is an equal number of A and B sites.

Definition 2.1: The Su–Schrieffer–Heeger (SSH) model

The Su-Schrieffer-Heeger model describes a dimerized chain of N unit cells with alterna1ng hopping magnitudes. The unit cell contains two sites (A and B sites), with intracell hopping v and intercell hopping ω. The Hamiltonian is given by

$$H = -\sum_{n=0} \Big(v\,|n;B\rangle\langle n;A| + v\,|n;A\rangle\langle n;B| + \omega\,|n+1;A\rangle\langle n;B| + \omega\,|n;B\rangle\langle n+1;A| \Big)$$

where n runs over all unit cells. In what follows, use $\omega = 1$ in your simulation.

Solution: We are going to define the Hamiltonian matrix for the SSH model in both periodic and open boundary conditions using the code block below:

```
# PBC
def pbc_Hamiltonian(n, nu, omega = 1):
    H = np.zeros((2 * n, 2 * n))
    for i in range(n):
        H[2 * i + 1, 2 * i] = -nu
        H[2 * i, 2 * i + 1] = -nu
        H[2 * (i + 1) % (2 * n), 2 * i + 1] = -omega
        H[2 * i + 1, 2 * (i + 1) % (2 * n)] = -omega
    return H

# OBC
def obc_Hamiltonian(n, nu, omega=1):
    H = np.zeros((2 * n, 2 * n))
    for i in range(n):
        H[2 * i + 1, 2 * i] = -nu
        H[2 * i, 2 * i + 1] = -nu
    for i in range(n - 1):
        H[2 * (i + 1), 2 * i + 1] = -omega
        H[2 * i + 1, 2 * (i + 1)] = -omega
    return H
```

Example 2.1 (SSH Hamiltonian in PBC and OBC)

```
# Example
N = 5
nu = 2
omega = 1
H_PBC = pbc_Hamiltonian(N, nu, omega)
H_OBC = obc_Hamiltonian(N, nu, omega)

print("Hamiltonian␣in␣PBC:")
print(H_PBC)
print(scipy.linalg.ishermitian(H_PBC))
print("Hamiltonian␣in␣OBC:")
print(H_OBC)
print(scipy.linalg.ishermitian(H_OBC))
```

whose outputs are as follows:

```
Hamiltonian in PBC:
[[ 0. -2.  0.  0.  0.  0.  0.  0.  0. -1.]
 [-2.  0. -1.  0.  0.  0.  0.  0.  0.  0.]
 [ 0. -1.  0. -2.  0.  0.  0.  0.  0.  0.]
 [ 0.  0. -2.  0. -1.  0.  0.  0.  0.  0.]
 [ 0.  0.  0. -1.  0. -2.  0.  0.  0.  0.]
 [ 0.  0.  0.  0. -2.  0. -1.  0.  0.  0.]
 [ 0.  0.  0.  0.  0. -1.  0. -2.  0.  0.]
 [ 0.  0.  0.  0.  0.  0. -2.  0. -1.  0.]
 [ 0.  0.  0.  0.  0.  0.  0. -1.  0. -2.]
 [-1.  0.  0.  0.  0.  0.  0.  0. -2.  0.]]
True
Hamiltonian in OBC:
[[ 0. -2.  0.  0.  0.  0.  0.  0.  0.  0.]
 [-2.  0. -1.  0.  0.  0.  0.  0.  0.  0.]
 [ 0. -1.  0. -2.  0.  0.  0.  0.  0.  0.]
 [ 0.  0. -2.  0. -1.  0.  0.  0.  0.  0.]
 [ 0.  0.  0. -1.  0. -2.  0.  0.  0.  0.]
 [ 0.  0.  0.  0. -2.  0. -1.  0.  0.  0.]
 [ 0.  0.  0.  0.  0. -1.  0. -2.  0.  0.]
 [ 0.  0.  0.  0.  0.  0. -2.  0. -1.  0.]
 [ 0.  0.  0.  0.  0.  0.  0. -1.  0. -2.]
 [ 0.  0.  0.  0.  0.  0.  0.  0. -2.  0.]]
True
```